

26. planner 指令の安全橋渡し

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/speed_command_bridge_node.py
種別	Python
区分	runtime node
役割	planner/speed_cmd と drive_mode を受け、起動直後や system state を見ながら安全な出力 topic/UDP に整形する。
起動文脈	実機へ出る直前のガード層。

このファイルを読む理由

planner/speed_cmd と drive_mode を受け、起動直後や system state を見ながら安全な出力 topic/UDP に整形する。

- rate limiter と command gate を持つ。
- planner の生指令をそのまま実車へは出さない。

呼び出し関係

どこから呼ばれるか

- mpc_solarcar/live_launch.py

次に読むと理解が進むファイル

- mpc_solarcar/solar_preflight_logic.py

同一リポジトリ内の主要依存

- mpc_solarcar/signal_utils.py
- mpc_solarcar/solar_preflight_logic.py
- mpc_solarcar/telemetry_protocol.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from .telemetry_protocol import utc_iso_now

class SpeedCommandBridgeNode(Node):
    def __init__(self) -> None:
        super().__init__("speed_command_bridge_node")
        self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
        self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")
        self.declare_parameter("udp_enabled", False)
        self.declare_parameter("udp_host", "127.0.0.1")
        self.declare_parameter("udp_port", 50050)
        self.declare_parameter("publish_rate_hz", 5.0)
        self.declare_parameter("input_timeout_sec", 3.0)
        self.declare_parameter("safe_speed_kmh", 0.0)
        self.declare_parameter("startup_hold_sec", 2.0)
        self.declare_parameter("filter_tau_sec", 1.0)
        self.declare_parameter("accel_limit_kmhps", 1.2)
        self.declare_parameter("decel_limit_kmhps", 3.5)
        self.declare_parameter("speed_deadband_kmh", 0.1)
        self.declare_parameter("speed_quantize_step_kmh", 0.1)
        self.declare_parameter("max_output_speed_kmh", 120.0)
        self.declare_parameter("drive_mode_min_hold_sec", 5.0)
```

主要構造の読み取り

主要クラスは SpeedCommandBridgeNode。主要関数は main。ROS パラメータ宣言は 18 件。ROS I/O は publisher 2 件、subscription 4 件。

主要クラス・関数

行	種別	名前	補足
17	class	SpeedCommandBridgeNode	bases: Node
18	function	__init__	args: self
89	function	_on_speed_cmd	args: self, msg
93	function	_on_upper_speed_cmd	args: self, msg
96	function	_on_drive_mode	args: self, msg
100	function	_on_system_state	args: self, msg
104	function	_step	args: self
162	function	main	

ファイルを上から読んだときの定義順

行	内容
---	----

17 クラス SpeedCommandBridgeNode を定義する。
162 関数 main を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>import json</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L153。
4	<code>import socket</code>	socket モジュールを利用するため。このファイル内での主な使用位置は L73。
5	<code>import time</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L74, L91, L98, L102, L105, L143。
7	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L163, L165, L167。
8	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L17。
10	<code>from std_msgs. msg import Float32, String</code>	Float32 や String など軽量メッセージで planner / vehicle 値を publish するため。このファイル内での主な使用位置は L66, L67, L68, L69, L70, L71, L89, L93, ...。
12	<code>from signal_ utils import SmoothRateLimiter</code>	signal_utils.py から SmoothRateLimiter を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/signal_utils.py。このファイル内での主な使用位置は L54。
13	<code>from solar_preflight_ logic import evaluate_ command_gate</code>	preflight 判定ロジックから evaluate_command_gate を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/solar_preflight_logic.py。このファイル内での主な使用位置は L107。
14	<code>from telemetry_ protocol import utc_iso_now</code>	telemetry_protocol.py から utc_iso_now を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/telemetry_protocol.py。このファイル内での主な使用位置は L144。

関数・クラスを上から順に解説

L17 クラス SpeedCommandBridgeNode

- 定義: SpeedCommandBridgeNode(bases=Node)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: Float32, SmoothRateLimiter, String, __init__, bool, create_publisher, create_subscription, create_timer, declare_parameter, dumps, encode, error
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。

1. 関数 __init__ を定義する。

2. 関数 `_on_speed_cmd` を定義する。
3. 関数 `_on_upper_speed_cmd` を定義する。
4. 関数 `_on_drive_mode` を定義する。
5. 関数 `_on_system_state` を定義する。
6. 関数 `_step` を定義する。

L162 関数 main

- 定義: `main()`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `SpeedCommandBridgeNode`, `destroy_node`, `init`, `shutdown`, `spin`
 - 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
1. `rclpy.init(...)` を実行する。
 2. `node` に `SpeedCommandBridgeNode()` の結果を代入する。
 3. `rclpy.spin(...)` を実行する。
 4. `node.destroy_node(...)` を実行する。
 5. `rclpy.shutdown(...)` を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
20	<code>output_speed_topic</code>	<code>/vehicle/speed_cmd_kmh</code>
21	<code>output_drive_mode_topic</code>	<code>/vehicle/drive_mode_cmd</code>
22	<code>udp_enabled</code>	<code>False</code>
23	<code>udp_host</code>	<code>127.0.0.1</code>
24	<code>udp_port</code>	<code>50050</code>
25	<code>publish_rate_hz</code>	<code>5.0</code>
26	<code>input_timeout_sec</code>	<code>3.0</code>
27	<code>safe_speed_kmh</code>	<code>0.0</code>
28	<code>startup_hold_sec</code>	<code>2.0</code>
29	<code>filter_tau_sec</code>	<code>1.0</code>
30	<code>accel_limit_kmhps</code>	<code>1.2</code>
31	<code>decel_limit_kmhps</code>	<code>3.5</code>
32	<code>speed_deadband_kmh</code>	<code>0.1</code>
33	<code>speed_quantize_step_kmh</code>	<code>0.1</code>
34	<code>max_output_speed_kmh</code>	<code>120.0</code>
35	<code>drive_mode_min_hold_sec</code>	<code>5.0</code>
36	<code>require_system_running</code>	<code>True</code>
37	<code>system_state_timeout_sec</code>	<code>2.5</code>

ROS topic I/O

行	種別	topic	callback / 補足
66	publisher	self.output_speed_topic	publisher
67	publisher	self.output_drive_mode_topic	publisher
68	subscription	/planner/speed_cmd	self._on_speed_cmd
69	subscription	/planner/upper_speed_cmd	self._on_upper_speed_cmd
70	subscription	/planner/drive_mode	self._on_drive_mode
71	subscription	/system/state	self._on_system_state

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/speed_command_bridge_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。